Table of Contents

1. Overview & Research Question

This project investigates the application of classical machine learning algorithms to phishing email detection — a critical challenge in applied cybersecurity. The work bridges Natural Language Processing (NLP) and information security, two fields with strong synergy in modern threat detection systems.

1.1 Research Question

"Can classical machine learning models effectively detect phishing emails, and how do different algorithms compare in precision and recall?"

1.2 Motivation

Phishing remains one of the most prevalent attack vectors in cybersecurity, responsible for over 80% of reported security breaches worldwide. Automated detection using ML offers scalability that rule-based systems cannot achieve. This project evaluates whether a simple, interpretable classifier can match or exceed rule-based baselines on a large real-world dataset.

✓ Research alignment: this type of applied ML security work is actively explored in adversarial ML, threat intelligence, and large-scale systems research.

1.3 Scope

This project covers binary classification of emails (phishing vs. legitimate) using TF-IDF text representation and two classical ML algorithms. It does not cover deep learning approaches or LLM-based classification — those are planned as next steps in the research roadmap.

2. Prerequisites

2.1 Required Tools

Tool	Version	Purpose	Install
Python	3.14+	Primary programming language	python.org/downloads
Git	2.54+	Version control	git-scm.com/downloads
VS Code	Latest	Code editor	code.visualstudio.com
Git Bash	Bundled	Unix-compatible terminal on Windows	Included with Git
Jupyter	via pip	Interactive notebook environment	pip install jupyter

Why Git Bash? Git Bash emulates a Unix shell on Windows, providing access to standard POSIX commands (mkdir, touch, ls, chmod, echo, curl). This mirrors the Linux/macOS environment used in most research and production systems, ensuring command compatibility across platforms.

✓ Unix-based tooling is the standard in research computing, cloud infrastructure, and open-source development. Mastering it early is a significant advantage.

2.2 Python Libraries

```
pip install pandas scikit-learn matplotlib seaborn jupyter kaggle
```

Library	Purpose
pandas	Data manipulation via DataFrames — reading CSVs, filtering, aggregating
scikit-learn	Machine learning algorithms, metrics, preprocessing utilities
matplotlib	Low-level plotting library — charts, figures, axes control
seaborn	High-level statistical visualization built on top of matplotlib
jupyter	Interactive notebook environment (.ipynb) for exploratory analysis
kaggle	Official Kaggle API client for programmatic dataset downloads

2.3 Git Identity Configuration

Before making any commit, configure your global Git identity. This information is embedded in every commit you create:

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

- *--global — applies to all repositories on this machine*
- *user.name — displayed in commit history and GitHub contributions graph*
- *user.email — must match your GitHub account email for attribution*

2.4 PATH Configuration in Git Bash

When pip installs packages on Windows, the executables land in a Scripts folder that Git Bash does not include in PATH by default. Add it manually:

```
export PATH="$PATH:/c/Users/YOUR_USER/AppData/Roaming/Python/Python314/Scripts"
```

- *export — sets an environment variable for the current shell session*
- *PATH — colon-separated list of directories the shell searches for executables*
- *$PATH — expands the current PATH value (we append, not replace)*

This command applies to the current terminal session only. To make it permanent, add it to ~/.bashrc

3. Project Structure

Standard ML project layout following industry conventions:

```
phishing-detection-ml/
■■■ data/               # Datasets (excluded from version control)
■■■ notebooks/          # Jupyter analysis notebooks
■   ■■■ analysis.ipynb
■■■ src/                # Modular Python source code (future)
■■■ .gitignore          # Files and folders excluded from Git
■■■ LICENSE             # MIT License
■■■ README.md           # Project documentation
```

■ *data/ — contains raw CSVs. Excluded via .gitignore (files exceed 100MB GitHub limit)*

■ *notebooks/ — exploratory analysis, modeling, and visualization*

■ *src/ — reserved for refactored, modular Python code in future iterations*

■ *.gitignore — tells Git which files to never track or commit*

3.1 Creating the Structure

```
mkdir data notebooks src
```

■ *mkdir — creates directories*

■ *In Git Bash, multiple directories can be created in a single command (Unix behavior)*

■ PowerShell requires different syntax: New-Item -ItemType Directory -Name data, notebooks, src. This is one reason Git Bash is preferred for cross-platform development workflows.

3.2 Why Not Version the Data?

GitHub enforces a 100MB per-file limit. ML datasets are typically larger. The industry standard is to version only code and document data provenance (origin, download instructions) in the README. The .gitignore handles this:

```
echo "data/" >> .gitignore
```

■ *echo "data/" — outputs the string to stdout*

■ *>> — redirects and appends stdout to the target file*

■ *.gitignore — Git reads this file to determine what to exclude from tracking*

4. Dataset

4.1 Source

We use the **Phishing Email Dataset** publicly available on Kaggle, published by Nasir Abdullah Alam. The dataset contains 82,486 real emails labeled as phishing (1) or legitimate (0), split nearly evenly between classes.

4.2 Downloading via Kaggle API

Step 1 — Install the library:

```
pip install kaggle
```

Step 2 — Generate your API token:

Go to kaggle.com → your profile → Settings → API → Create New Token. A kaggle.json file will be downloaded containing your credentials.

■ NEVER expose your API token in public chats, commits, or files. An exposed token is a compromised credential. Revoke it immediately if leaked.

Step 3 — Configure the token:

```
mkdir -p ~/.kaggle && echo YOUR_TOKEN > ~/.kaggle/access_token && chmod 600 ~/.kaggle/acce
ss_token
```

■ *mkdir -p ~/.kaggle — creates the config directory; -p suppresses errors if it exists*

■ *echo TOKEN > file — writes the token string into the file*

■ *chmod 600 — Unix permission: owner can read/write, no access for others*

■ *~/.kaggle — hidden directory in the home folder (~) for Kaggle config*

Step 4 — Download the dataset:

```
kaggle datasets download -d naserabdullahalam/phishing-email-dataset -p data/ --unzip
```

■ *datasets download — Kaggle CLI subcommand for dataset downloads*

■ *-d — specifies the dataset by its Kaggle slug (owner/dataset-name)*

■ *-p data/ — sets the download destination directory*

■ *--unzip — automatically extracts the downloaded archive*

4.3 Available Files

File	Rows	Columns
phishing_email.csv	82,486	text_combined, label
Enron.csv	720,544	subject, body, label
CEAS_08.csv	1,305,707	sender, receiver, date, subject, body, label, urls
SpamAssasin.csv	201,451	sender, receiver, date, subject, body, label, urls
Nigerian_Fraud.csv	156,001	sender, receiver, date, subject, body, urls, label
Ling.csv	5,476	subject, body, label

Nazario.csv	2,794	sender, receiver, date, subject, body, urls, label

✓ We use phishing_email.csv as the primary dataset: text is pre-combined into a single column (text_combined), simplifying the preprocessing pipeline.

5. Preprocessing

5.1 Loading and Initial Inspection

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('../data/phishing_email.csv')
df.head()
```

■ *import — loads the library into the current namespace*

■ *as pd / plt / sns — aliases for convenience (shorter to type)*

■ *pd.read_csv — parses a CSV file into a DataFrame*

■ *df.head() — displays the first 5 rows for a quick sanity check*

```
print(df.shape)                    # rows and columns
print(df['label'].value_counts())  # class distribution
print(df.isnull().sum())           # missing values per column
```

■ *df.shape — returns (82486, 2): 82,486 rows, 2 columns*

■ *value_counts() — 42,891 phishing (1), 39,595 legitimate (0)*

■ *isnull().sum() — returns 0 for both columns: no missing data*

✓ Dataset is clean and nearly balanced — ideal conditions for training a classifier.

5.2 TF-IDF Vectorization

ML algorithms require numerical input. **TF-IDF (Term Frequency — Inverse Document Frequency)** converts each email into a numerical vector where each dimension represents the weighted importance of a word.

TF (Term Frequency): how often a word appears in a given document. **IDF (Inverse Document Frequency):** how rare the word is across all documents — common words like 'the' receive low weight; rare, discriminative words receive high weight.

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=5000, stop_words='english')

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf  = vectorizer.transform(X_test)
```

■ *max_features=5000 — retains only the 5,000 most informative terms*

■ *stop_words='english' — removes high-frequency filler words (the, a, is, ...)*

■ *fit_transform — learns vocabulary from training data AND transforms it*

■ *transform — applies the learned vocabulary to test data (no re-learning)*

■ Always fit only on training data. Fitting on the full dataset causes data leakage — the model would have indirect knowledge of the test set.

5.3 Train/Test Split

```
from sklearn.model_selection import train_test_split

X = df['text_combined']
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

■ *X — input features (email text)*

■ *y — target labels (0 = legitimate, 1 = phishing)*

■ *test_size=0.2 — 20% reserved for evaluation (16,498 emails)*

■ *random_state=42 — fixed seed ensures reproducible splits across runs*

✓ Reproducibility is a core principle in research: anyone running the same code must obtain identical results.

6. Machine Learning Models

6.1 Logistic Regression

Despite its name, Logistic Regression is a **classification** algorithm. It learns a weight for each feature (word) and computes the probability that an email is phishing using the logistic (sigmoid) function. It is interpretable, efficient, and performs well on high-dimensional sparse data like TF-IDF vectors.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

model = LogisticRegression(max_iter=1000)
model.fit(X_train_tfidf, y_train)

y_pred = model.predict(X_test_tfidf)
print(classification_report(y_test, y_pred))
```

- *max_iter=1000 — maximum iterations for the solver to converge*
- *fit — trains the model by adjusting weights to minimize classification error*
- *predict — applies the trained model to the test set*
- *classification_report — prints precision, recall, F1, and support per class*

6.2 Naive Bayes (Multinomial)

Multinomial Naive Bayes is a probabilistic classifier based on Bayes' Theorem. It assumes feature independence (the 'naive' assumption) and estimates the probability of each class given the observed word counts. Historically one of the most effective algorithms for text classification and spam filtering.

```
from sklearn.naive_bayes import MultinomialNB

model_nb = MultinomialNB()
model_nb.fit(X_train_tfidf, y_train)

y_pred_nb = model_nb.predict(X_test_tfidf)
print(classification_report(y_test, y_pred_nb))
```

- *MultinomialNB — variant of Naive Bayes suited for discrete count data (word frequencies)*
- *No hyperparameters needed — the algorithm is parameter-light by design*

7. Results & Analysis

7.1 Evaluation Metrics

Metric	Definition	Security relevance
Accuracy	Overall percentage of correct classifications	General model health
Precision	Of all predicted phishing, how many truly were?	Minimizes false alarms
Recall	Of all real phishing, how many were detected?	Critical: catch everything
F1-Score	Harmonic mean of Precision and Recall	Balance between both

7.2 Model Comparison

Model	Accuracy	Precision	Recall	F1	False Negatives
Logistic Regression	**98%**	**98%**	**98%**	**98%**	**132**
Naive Bayes	96%	96%	96%	96%	~428

7.3 Confusion Matrix — Logistic Regression

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Legitimate', 'Phishing'],
            yticklabels=['Legitimate', 'Phishing'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix — Logistic Regression')
plt.show()
```

■ *confusion_matrix — computes a 2x2 matrix of TP, TN, FP, FN counts*

■ *sns.heatmap — renders the matrix as a color-coded grid*

■ *annot=True — displays the count inside each cell*

■ *fmt='d' — formats numbers as integers (not scientific notation)*

	Predicted: Legitimate	Predicted: Phishing
Actual: Legitimate	7,744 (TN) ✓	191 (FP) — false alarm
Actual: Phishing	132 (FN) — missed	8,431 (TP) ✓

False Negatives (132) are the most critical error in security contexts: a phishing email classified as legitimate reaches the user's inbox. At only 132 out of 8,563 phishing emails (1.5%), the model demonstrates strong real-world viability.

8. Git & GitHub

8.1 Creating the Remote Repository

On github.com: click + → New repository → name: phishing-detection-ml → set to Public → check Add README file → Create repository.

8.2 Cloning Locally

```
git clone https://github.com/YOUR_USER/phishing-detection-ml.git .
```

■ *git clone — copies the remote repository to the local machine*

■ *URL.git — the HTTPS address of the GitHub repository*

■ *. (dot) — clones into the current directory instead of creating a subdirectory*

8.3 Standard Git Workflow

```
git add .                                    # stage all modified files
git commit -m "feat: description of change"  # record snapshot with message
git push                                     # sync to GitHub
```

■ *git add . — moves all changes from working directory to the staging area*

■ *git commit -m — creates a permanent snapshot with a descriptive message*

■ *git push — uploads local commits to the remote repository on GitHub*

✓ Conventional Commits format for messages: feat: (new feature), fix: (bug fix), docs: (documentation), refactor: (code restructure)

8.4 Undoing a Commit

```
git reset HEAD~1
```

■ *reset — moves the branch pointer back in history*

■ *HEAD~1 — one commit behind the current HEAD*

■ *Files are NOT deleted — only the commit record is undone*

8.5 Excluding Large Files

```
echo "data/" >> .gitignore
git add .gitignore
git commit -m "chore: exclude data directory from version control"
```

■ GitHub rejects files over 100MB. Always add large data directories to .gitignore before the first commit. Removing them after the fact is complex.

9. Research README

The README is the first thing any technical reviewer reads. For an ML project, it should communicate: what problem was investigated, how it was approached, and what was found — structured as a concise paper abstract.

```
# Phishing Detection: ML Classifiers vs Rule-Based Approaches

## Research Question
Can classical ML models effectively detect phishing emails,
and how do different algorithms compare in precision and recall?

## Dataset
- Source: Phishing Email Dataset (Kaggle)
- Size: 82,486 emails | 52% phishing, 48% legitimate

## Models
| Model               | Accuracy | F1  |
|---------------------|----------|-----|
| Logistic Regression | 98%      | 98% |
| Naive Bayes         | 96%      | 96% |

## Key Findings
- Logistic Regression outperforms Naive Bayes by 2% across all metrics
- False negative rate below 2% (132 out of 8,563 phishing emails)
- TF-IDF with 5,000 features provides strong text representation

## Tech Stack
Python 3.14 · scikit-learn · pandas · matplotlib · seaborn

## Author
Guilherme U Pereira
B.S. Student — Artificial Intelligence & Blockchain and Digital Cryptography, FMU
Cisco Cybersecurity Week 2026 — Selected Participant
Cisco Cyber Education Program — Scholarship Holder
```

10. Glossary

Term	Definition
DataFrame	Tabular data structure in pandas. Rows are records, columns are attributes.
Feature	Input variable to the model. Here: individual words from email text.
Label	Output variable. Here: 0 (legitimate) or 1 (phishing).
TF-IDF	Numerical text representation weighting words by frequency and rarity.
Train/Test Split	Dataset partition: train set to learn, test set to evaluate generalization.
Overfitting	Model memorizes training data but fails on unseen data. Mitigated by splitting.
Precision	Of all predicted positives, what fraction were truly positive.
Recall	Of all actual positives, what fraction the model correctly identified.
F1-Score	Harmonic mean of Precision and Recall. Balances both metrics.
Confusion Matrix	2x2 table showing TP, TN, FP, FN counts for a binary classifier.
False Negative	Phishing email classified as legitimate. Most critical error type.
False Positive	Legitimate email classified as phishing. Causes unnecessary alerts.
random_state	Fixed integer seed for random number generation. Ensures reproducibility.
fit / transform	fit: learn parameters from data. transform: apply learned parameters.
data leakage	When test data information influences training. Invalidates evaluation.
.gitignore	Text file listing patterns of files Git should never track.
HEAD	Pointer to the current commit in the Git history.
PATH	Shell variable listing directories searched for executable commands.
chmod 600	Unix permission: owner read/write only; no access for group or others.
slug	URL-friendly identifier. Kaggle slugs follow owner/dataset-name format.